MinhaLoja

Como o código da loja virtual funciona — HTML, CSS, JavaScript e Supabase

1. Visão geral

A loja é um site de vendas simples, mas completo: mostra produtos vindos de um banco de dados, permite buscar e filtrar por categoria, montar um carrinho e finalizar o pedido. O pedido é gravado no **Supabase**, um serviço que oferece um banco **PostgreSQL** acessível direto do navegador.

O projeto tem quatro arquivos, cada um com uma responsabilidade:

Arquivo	Papel
index.html	Estrutura da página: cabeçalho, banner, vitrine, carrinho e formulário de checkout.
style.css	Aparência: cores, tamanhos, layout responsivo (computador e celular).
app.js	Comportamento: conecta ao Supabase, lê produtos, controla carrinho e envia pedidos.
supabase.sql	Cria as tabelas, as regras de segurança e a função que registra pedidos.

O caminho dos dados é sempre o mesmo: **navegador** → **biblioteca supabase-js** → **API do Supabase** → **PostgreSQL**. O site nunca fala com o banco "por fora" das regras: tudo passa pela segurança configurada no SQL.

2. Colocando para funcionar (passo a passo)

2.1 Criar o projeto no Supabase

- Acesse supabase.com, crie uma conta e clique em **New project**.
- Escolha um nome, uma senha para o banco e a região mais próxima (ex.: São Paulo).
- Aguarde o projeto ficar pronto (leva cerca de 1–2 minutos).

2.2 Rodar o SQL

- No menu lateral, abra **SQL Editor** → **New query**.
- Cole todo o conteúdo de **supabase.sql** e clique em **Run**.
- Abra o **Table Editor**: devem aparecer as tabelas categorias, produtos, pedidos e itens_pedido, já com 7 produtos de exemplo.

2.3 Pegar as chaves e colar no app.js

Em **Project Settings** → **API** (em versões novas do painel pode aparecer como *Data API* / *API Keys*), copie:

- **Project URL** — o endereço do projeto, algo como https://abcdefgh.supabase.co
- **anon public key** (ou *publishable key*) — a chave pública.

Cole as duas no início do **app.js**:

```
const SUPABASE_URL = "https://abcdefgh.supabase.co";
const SUPABASE_ANON_KEY = "eyJhbGciOiJIUzI1NiIs...";
```

Importante: a *anon key* é pública e pode ficar no navegador — quem protege os dados são as regras RLS do SQL. **Nunca** coloque no site a chave *service_role* (ou *secret key*): ela ignora todas as regras e dá acesso total ao banco.

2.4 Abrir o site

Abra o **index.html** no navegador. Para evitar bloqueios de alguns navegadores com arquivos locais, o ideal é usar um servidor simples, como a extensão **Live Server** do VS Code ou o comando:

```
npx serve .
```

Enquanto as chaves não forem preenchidas, a loja roda em **modo demonstração**, com produtos de exemplo e um aviso laranja no topo. Assim você consegue ver o visual antes mesmo de criar o banco.

3. O HTML (index.html)

O HTML só descreve **o que existe** na página. Nada nele é "inteligente": as partes que mudam (produtos, categorias, itens do carrinho) começam vazias e são preenchidas pelo JavaScript. Para isso, cada uma tem um **id** que o JS usa para encontrá-la:

id	O que é
grade-produtos	Onde os cartões dos produtos são desenhados.
lista-categorias	Botões "Todas", "Eletrônicos", "Casa"...
campo-busca	Campo de pesquisa do cabeçalho.
carrinho / itens-carrinho	Painel lateral e a lista de itens dentro dele.
modal-checkout / form-checkout	Janela com o formulário de dados do cliente.
alerta-conexao	Aviso de modo demonstração ou de erro de conexão.
toast	Mensagem rápida que aparece no rodapé da tela.

No final do **body** são carregados dois scripts, nesta ordem: primeiro a biblioteca oficial do Supabase (pelo CDN jsDelivr) e depois o **app.js**, que depende dela.

4. O CSS (style.css) e as cores

Todas as cores ficam reunidas como **variáveis CSS** no topo do arquivo, dentro de **:root**. Para mudar a identidade visual da loja inteira basta trocar esses valores:

	Variável	Cor	Uso
	`--cor-primaria`	`#FF6B00`	Laranja — botões, preços, "compre agora"
	`--cor-secundaria`	`#1E2A3A`	Azul-marinho — cabeçalho e rodapé (confiança)
	`--cor-destaque`	`#FFC400`	Amarelo — selos de desconto e ofertas
	`--cor-sucesso`	`#1FA35B`	Verde — mensagens de sucesso
	`--cor-erro`	`#D93636`	Vermelho — erros e "esgotado"

A escolha segue o padrão do varejo online: o **laranja** chama a atenção para a ação de compra, o **azul-escuro** passa segurança e o **amarelo** destaca promoções.

Técnicas usadas

- **Grid responsivo**: `repeat(auto-fill, minmax(220px, 1fr))` cria quantas colunas couberem na tela, sem precisar calcular nada.
- **Cabeçalho fixo**: `position: sticky` mantém a busca e o carrinho sempre visíveis.
- **Carrinho deslizante**: o painel começa com `transform: translateX(100%)` (fora da tela). Quando o JS adiciona a classe **aberto**, ele volta para 0 com uma transição suave.
- **Media query** (`max-width: 640px`): no celular, a busca desce para outra linha e a vitrine fica com 2 colunas.

5. O JavaScript (app.js)

O arquivo está dividido em 12 blocos numerados e comentados. Os principais:

5.1 Configuração e conexão (blocos 1 e 2)

```
const configurado = SUPABASE_URL.startsWith("http")
                 && !SUPABASE_ANON_KEY.startsWith("COLE_AQUI");
const banco = configurado
  ? window.supabase.createClient(SUPABASE_URL, SUPABASE_ANON_KEY)
  : null;   // null = modo demonstração
```

Se as chaves foram preenchidas, **createClient** cria o objeto **banco**, que será usado para todas as consultas. Caso contrário, **banco** fica *null* e a loja usa os dados de demonstração.

5.2 Estado da aplicação (bloco 4)

Algumas variáveis guardam a "situação atual": a lista de produtos, a categoria selecionada, o texto da busca e o carrinho. Sempre que algo muda, uma função de **renderizar** redesenha a parte da tela correspondente a partir desse estado. Esse padrão (estado → tela) é a mesma ideia usada por frameworks como React.

5.3 Lendo dados do banco (bloco 7)

```
const [respCategorias, respProdutos] = await Promise.all([
  banco.from("categorias").select("id, nome").order("nome"),
  banco.from("produtos")
       .select("id, nome, descricao, preco, ...")
       .eq("ativo", true)
       .order("criado_em", { ascending: false }),
]);
```

A biblioteca traduz essa cadeia de métodos em SQL. A segunda consulta equivale a `SELECT ... FROM produtos WHERE ativo = true ORDER BY criado_em DESC`. O **Promise.all** faz as duas consultas ao mesmo tempo, deixando o carregamento mais rápido.

O Supabase não lança exceções: ele devolve `{ data, error }`. Por isso o código verifica **error** e, se houver problema, mostra um aviso e cai nos produtos de exemplo, para o site nunca ficar em branco.

5.4 Desenhando a vitrine (bloco 8)

renderizarProdutos() filtra a lista pela categoria e pela busca (sem diferenciar acentos, graças à função **normalizar**), calcula o percentual de desconto e monta o HTML de cada cartão com *template strings*. Todo texto vindo do banco passa por **escaparHTML()**, que impede que um nome de produto malicioso execute código na página (ataque XSS).

5.5 Carrinho (bloco 9)

- O carrinho é um array de objetos `{ id, nome, preco, imagem_url, quantidade }`.
- É salvo no **localStorage**, então não se perde se a página for recarregada.
- Antes de aumentar a quantidade, o código compara com o **estoque** do produto.
- Toda alteração chama **salvarCarrinho()**, que grava e redesenha o painel e o contador do cabeçalho.

5.6 Eventos com delegação (bloco 11)

Os botões de "Adicionar ao carrinho" são criados dinamicamente, então não dá para ligar um evento em cada um quando a página abre. A solução é colocar **um único ouvinte** no elemento pai e descobrir qual botão foi clicado pelo atributo **data-***:

```
el.grade.addEventListener("click", (e) => {
  const botao = e.target.closest("[data-adicionar]");
  if (botao) adicionarAoCarrinho(Number(botao.dataset.adicionar));
});
```

5.7 Finalizando o pedido (bloco 10)

```
const { data: idPedido, error } = await banco.rpc("criar_pedido", {
  p_cliente_nome: dados.get("nome"),
  p_cliente_email: dados.get("email"),
  p_cliente_telefone: dados.get("telefone"),
  p_endereco: dados.get("endereco"),
  p_itens: [{ produto_id: 1, quantidade: 2 }, ...],
});
```

Repare que o site envia apenas **o ID e a quantidade** de cada produto — **nunca o preço**. O **.rpc()** chama a função **criar_pedido** que está dentro do banco, e é ela quem busca o preço real. Assim, mesmo que alguém altere o preço pelo console do navegador, o pedido será gravado com o valor correto.

6. O banco de dados (supabase.sql)

6.1 Tabelas e relacionamentos

Tabela	Guarda	Ligações
categorias	id, nome	—
produtos	nome, descrição, preço, preço antigo, estoque, imagem, ativo	categoria_id → categorias
pedidos	dados do cliente, endereço, total, status	—
itens_pedido	quantidade e preço unitário de cada produto comprado	pedido_id → pedidos produto_id → produtos

Um pedido tem vários itens (relação 1-para-N). O **preco_unitario** é copiado no momento da compra: se o preço do produto mudar depois, o histórico do pedido continua correto. Restrições **check** garantem regras básicas direto no banco, como estoque nunca negativo e status apenas entre *pendente, pago, enviado, entregue, cancelado*.

6.2 Segurança com RLS (Row Level Security)

Quando o RLS é ligado numa tabela, **nada** é permitido até que uma *policy* (regra) libere. O site entra no banco com o papel **anon** (por causa da anon key). As regras criadas são:

- **categorias**: anon pode ler tudo.
- **produtos**: anon pode ler apenas produtos com `ativo = true`.
- **pedidos** e **itens_pedido**: nenhuma regra para anon. Ninguém consegue ler, alterar ou apagar pedidos pela anon key — os dados pessoais dos clientes ficam protegidos.

Você, como dono, vê os pedidos pelo painel do Supabase, que usa permissões de administrador.

6.3 A função criar_pedido

Se o público não pode gravar em **pedidos**, como o site registra uma compra? Pela função **criar_pedido**, declarada com **security definer**: ela roda com a permissão do dono do banco, mas só faz exatamente o que está programado nela. É uma "porta controlada". Passo a passo:

- Valida nome, formato do e-mail, endereço e se o carrinho não está vazio.
- Cria o registro em **pedidos**.
- Para cada item: busca o produto com `FOR UPDATE` (trava a linha para que duas pessoas não comprem o último item ao mesmo tempo), confere o estoque, grava o item com o **preço do banco** e desconta o estoque.
- Soma o total e atualiza o pedido.
- Devolve o número do pedido para o site mostrar ao cliente.

Como tudo acontece dentro de uma única função, vale a regra do **tudo ou nada** (transação): se qualquer item falhar — por exemplo, estoque insuficiente — o PostgreSQL desfaz tudo, e nenhum pedido pela metade fica gravado. A linha `set search_path = public` é uma proteção recomendada para funções *security definer*.

> O SQL foi testado num PostgreSQL 16: roda mais de uma vez sem erro, cria o pedido com o total correto, desconta o estoque, recusa compras acima do estoque e impede o papel anon de ler a tabela de pedidos.

7. O fluxo completo de uma compra

1. A página abre e **iniciar()** busca categorias e produtos no Supabase.
2. O cliente filtra, busca e clica em **Adicionar ao carrinho**; o item vai para o array e para o localStorage.
3. No carrinho, ajusta quantidades e clica em **Finalizar compra**.
4. Preenche o formulário; o JS chama **banco.rpc("criar_pedido", ...)**.
5. O banco valida, grava pedido e itens, baixa o estoque e devolve o número do pedido.

6. O site mostra "Pedido nº X realizado", esvazia o carrinho e recarrega os produtos com o estoque atualizado.

8. Como personalizar e próximos passos

- **Trocar cores**: edite as variáveis em `:root` no style.css.
- **Trocar nome e textos**: edite o index.html (logo, banner, barra de aviso).
- **Cadastrar produtos**: pelo Table Editor do Supabase. Para imagens próprias, use o **Storage** do Supabase e cole a URL pública em *imagem_url*.
- **Esconder um produto** sem apagá-lo: mude *ativo* para false.
- **Publicar**: os arquivos são estáticos, então funcionam em Netlify, Vercel ou GitHub Pages.
- **Evoluções possíveis**: login de clientes com Supabase Auth (para ver "meus pedidos"), pagamento real com Mercado Pago ou Stripe, e uma página de administração protegida.